

Article

Machine Learning Models for SQL Injection Detection

Cosmina-Mihaela Rosca ¹, Adrian Stancu ^{2,*} and Catalin Popescu ^{2,*}

- ¹ Department of Automatic Control, Computers, and Electronics, Faculty of Mechanical and Electrical Engineering, Petroleum-Gas University of Ploiesti, 100680 Ploiesti, Romania; cosmina.rosca@upg-ploiesti.ro
- ² Department of Business Administration, Faculty of Economic Sciences, Petroleum-Gas University of Ploiesti, 100680 Ploiesti, Romania
- * Correspondence: astancu@upg-ploiesti.ro (A.S.); cpopescu@upg-ploiesti.ro (C.P.)

Abstract

Cyberattacks include Structured Query Language Injection (SQLi), which represents threats at the level of web applications that interact with the database. These attacks are carried out by executing SQL commands, which compromise the integrity and confidentiality of the data. In this paper, a machine learning (ML)-based model is proposed for identifying SQLi attacks. The authors propose a two-stage personalized software processing pipeline as a novel element. Although individual techniques are known, their structured combination and application in this context represent a novel approach to transforming raw SQL queries into input features for an ML model. In this research, a dataset consisting of 90,000 SQL queries was constructed, comprising 17,695 legitimate and 72,304 malicious queries. The dataset consists of synthetic data generated using the GPT-4o model and data from a publicly available dataset. These were processed within a pipeline proposed by the authors, consisting of two stages: syntactic normalization and the extraction of the eight semantic features for model training. Also, within the research, several ML models were analyzed using the Azure Machine Learning Studio platform. These models were paired with different sampling algorithms for selecting the training set and the validation set. Out of the 15 training-sampling algorithm combinations, the Voting Ensemble model achieved the best performance. It achieved an accuracy of 96.86%, a weighted AUC of 98.25%, a weighted F1-score of 96.77%, a weighted precision of 96.92%, and a Matthews correlation coefficient of 89.89%. These values demonstrate the model's ability to classify queries as legitimate or malicious. The attack identification rate was only 15 malicious queries missed out of a total of 7200, and the number of false alarms was 211 cases. The results confirm the possibility of integrating this algorithm into an additional security layer within an existing web application architecture. In practice, the authors suggest adding an extra layer of security using synthetic data.

Keywords: cybersecurity; web application security; synthetic data generation; feature engineering; syntactic normalization; semantic features; false positive reduction; query preprocessing



Academic Editors: Aryya Gangopadhyay and Wajeb Gharibi

Received: 12 July 2025

Revised: 17 August 2025

Accepted: 25 August 2025

Published: 27 August 2025

Citation: Rosca, C.-M.; Stancu, A.; Popescu, C. Machine Learning Models for SQL Injection Detection. *Electronics* **2025**, *14*, 3420. <https://doi.org/10.3390/electronics14173420>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Modern society frequently utilizes artificial intelligence (AI) tools that raise concerns about data privacy. This objective is addressed by the General Data Protection Regulation (GDPR) and represents one of the main concerns of the right to privacy. In this context, Structured Query Language (SQL) Injection (SQLi) is one of the most common security vulnerabilities that manifest through web applications. This vulnerability allows access to

the personal data of users of a web application. SQLi is listed at the top of the Open Web Application Security Project (OWASP) and is considered one of the most significant security risks. This vulnerability manifests when an application interacts with a relational database in an insecure manner. Essentially, the web application developers failed to integrate the necessary mechanisms to prevent an attack that injects malicious SQL commands into legitimate queries. Thus, SQLi attacks occur when the input allows the user to enter content that is not validated before being directly included in an SQL query.

An experienced attacker knows how to query and inspect a web application sufficiently to identify vulnerabilities that can manipulate query behavior through unauthorized access to sensitive user data or even the platform itself. In this way, the attacker can modify the database content by executing administrative commands at the server level. From a technical standpoint, SQLi represents an input validation vulnerability. This type of vulnerability is based on the lack of a query preprocessing mechanism, such as prepared statements or stored procedures. The impact ranges from the vulnerability of confidential information to the complete compromise of the application's infrastructure, making SQLi prevention a major concern in cybersecurity discussions regarding the software development process.

In this research, the aim is to examine machine learning (ML) methods as an intermediate layer between input retrieval and query execution, without excluding the classic security layer, ORM, Prepared Statement, Store Procedure, or other SQLi prevention techniques from this ensemble. The authors further recommend the continued use of previously employed security layers, with the introduction of an additional layer that integrates ML components as a second measure to prevent SQLi. In this way, the authors propose training an ML model that captures a series of input features. For training the model, the following research questions (RQs) are posed:

1. How is the dataset constructed for training the model in the context of data privacy compliance?
2. What are the features used in training the model so that it can distinguish between malicious and legitimate queries?
3. Is it feasible to integrate a secondary security layer that uses specific ML methods?
4. How do the metrics obtained from the model relate to practical use?

In this research, these RQs will be addressed as part of a protocol for preventing cyberattacks on relational databases.

The authors' contributions in this research address the following aspects:

1. In this research, a software pipeline consisting of two stages was developed. The first normalizes SQL queries at both the syntactic and semantic levels by replacing customized values with standard expressions. In contrast, the second is related to feature extraction, which enables the ML model to generalize to detect SQLi attacks.
2. This research proposed a method for generating a synthetic dataset correlated with a public dataset to obtain 90,000 queries. The synthetic dataset was generated using the GPT-4o model. The use of the synthetic dataset excludes the ethical issues associated with using real data.
3. The authors proposed eight repetitive features within SQL queries at the semantic and structural level. These characteristics allow differentiation between legitimate and malicious queries.
4. This study trained and evaluated ML algorithms by analyzing their performance. In the research, five ML algorithms were analyzed in conjunction with three sampling algorithms to perform the selection of training data versus validation data.
5. This study identified the model with the best performance metrics and integrated it into a security architecture applied within web applications as an additional layer to traditional protection mechanisms.

The paper is structured into six sections. Section 2 focuses on SQLi methods in the specialized literature, whereas Section 3 is dedicated to the materials and methods employed in building the dataset, the description of the implemented algorithms, and the technologies involved in data analysis, processing, and the construction of the proposed software pipeline, as well as the training and evaluation of ML models. Section 4 presents the research results, while Sections 5 and 6 are dedicated to the discussion and conclusions of the research, respectively.

2. Literature Review of SQL Injection Through ML

The detection of SQLi vulnerabilities is carried out in the specialized literature using various techniques. Alhowiti and Mohamed [1] classify existing detection solutions into prevention methods based on input validation and parameterization through Object-Relational Mapping (ORM) techniques and stored procedures. A second approach targets defense mechanisms, such as runtime detection. For example, Jang [2] addresses the detection of SQLi vulnerabilities in the context of SQL queries directly embedded in the programming language. This is an area not covered by traditional methods, proposing the generation of candidate code that detects SQL injections in the source code at runtime [3]. Yuan et al. [4] introduce a tool based on static analysis for applications that use object-oriented database extensions. The method involves code transformation, meaning the conversion of object-oriented code into equivalent procedural code and detection using control flow graphs and tight analysis. Erdődi et al. [5] propose modeling SQLi exploitation using Reinforcement Learning (RL). The problem is formulated as a Markov Decision Process, and the RL agents are trained to learn a general attack policy. The research direction continues through a varied range of SQLi vulnerabilities that demonstrate the ability to transfer knowledge between different typologies. Abikoye et al. [6] use an SQLi detection system based on the Knuth–Morris–Pratt (KMP) string search algorithm. An application-level implementation of PHP with MySQL using SQLMap in a virtualized Linux environment is presented in the paper by Bedeković et al. [7], which aims to demonstrate the execution of the attack. The prioritization of tests for SQLi is presented in the paper by Yang et al. [8], where the defense vectors are adjusted for subsequent tests.

The security of web applications is studied through SQLi attacks, as they represent some of the most dangerous threats to data protection through different AI methods [9–12]. In the specialized literature, SQLi attacks are studied using ML algorithms to detect potential security breaches. Identifying repetitive features is one of the challenges faced by ML models in the context of SQLi.

Arasteh et al. [13,14] used feature selection methods combined with classifiers such as Artificial Neural Networks. Thus, in the paper [13], the authors achieved an accuracy of 99.35%, while in the research [14], the authors obtained an accuracy of 99.68%, for a dataset consisting of 13 features. The two studies focus on optimizing the features used in training with ML algorithms. Le et al. [15] evaluate multiple ML algorithms, achieving an accuracy of 99.50% for Random Forest (RF) and Adaptive Boosting (AdaBoost). Peralta-Garcia et al. [16] compared and identified an accuracy of 99% for RF and a precision of 98%. Banimustafa et al. [17] also investigate RF and achieve an accuracy of 98% in identifying SQLi attacks.

Other research addresses deep learning (DL) models for automated feature engineering [18]. For example, Thalji et al. [19] combine the autoencoder model with Extreme Gradient Boosting (XGBoost) to achieve an accuracy of 99%, while Alghawazi et al. [20] employ a Recurrent Neural Network (RNN) autoencoder to achieve an accuracy of only 94% in SQLi detection. The Probabilistic Neural Network (PNN) model is also investigated for the same purpose, achieving an accuracy of 99.19% [21]. Other techniques achieved

an accuracy of 98.02% in the paper by Muduli et al. [22] and over 99.8% in the paper by Bakır [23]. There are researchers who use Natural Language Processing (NLP) [24,25] or combinations of models such as CNN plus Long Short-Term Memory (LSTM), achieving an accuracy of 99.84% [26]. Additionally, the combination of a Convolutional Neural Network (CNN) with LSTM and multiclass classification led to an F1-score of 97% in the paper by Paul et al. [27].

Sun et al. [28] propose another combination of models that leads to a declared F1-score accuracy of 95.64%. Farooq [10] achieved an accuracy of 99.33% for SQLi detection using the Light Gradient Boosting Machine (LightGBM) model and 99.11% for AdaBoost. Logistic Regression (LR) is studied in the research by Crespo-Martínez et al. [29], achieving an accuracy of 97% [30].

The security of SQL queries is studied by Alqhtani et al. [31] in the form of an adversarial proposal. In the papers by Demetrio et al. [32] and Valenza et al. [33], the SQLi problem is investigated using adversarial tools that deceive ML Web Application Firewalls (WAFs), and in the paper by Zuech et al. [34], Feature Selection Techniques (FSTs) are proposed to simplify models without performance loss. SQLi is also studied in the context of Cloud, IoT, and Edge Computing technologies [35–39]. These papers report an accuracy of 99.07% [37] and 94% [38] in detecting several multilayer attacks.

Other approaches target the variety of datasets [40] or testing on models with new distributions (cross-validation) [41]. Maruthavani and Shantharajah [42] use Spark and Fuzzy Neural Networks (FNNs) for SQLi detection in biomedical data. Additionally, the research by Ahmed et al. [43] builds the PhishCatcher extension with RF, achieving 98.5% accuracy. Other datasets target NoSQL injection approaches (MongoDB) [44] or propose blockchain for medical data protection [45,46].

The detection of SQLi attacks by reducing the false alarm rate is achieved through data mining and ML algorithms that use the CountVectorizer technique for feature extraction, chi-square feature selection, and a proprietary model called Performance Analysis and Iterative Optimization of the SQLi Detection Model from the paper by Ashlam et al. [47]. The model proposed in this paper improves detection from 94% to 99%. The method identified by Zhao et al. [48] is based on parsing the code into an Abstract Syntax Tree (AST), then transforming it into a dependency graph, with vectorization for model training.

SQLi detection can also be approached from the perspective of anomaly detection [49]. This review paper, along with the paper by Ahmad et al. [50], conducts a meta-analysis on the detection of phishing attacks. Alongside these review-type approaches, Janabi et al. [51] investigate security in Software-Defined Networking (SDN) networks, which are vulnerable to SQLi attacks, proposing a strategic framework for real-time traffic processing. The research by Zuech et al. [52] analyzes the issue of positive class rarity (attacks) in security data. This paper investigates three types of attacks: Brute Force, XSS, and SQLi, using random undersampling (RUS) in various proportions to balance the data. The best result obtained in this paper is reported for RF [53].

Analyzing the existing studies in the literature, it is found that the accuracy achieved in the proposed models for SQLi attack detection is high, reaching up to 99.8%. Although these values demonstrate the possibility of integrating the model in practice, most research relies on controlled datasets, which have well-defined scenarios and lead to limitations in applicability in realistic contexts, especially since these datasets use data that do not always have usage rights. Given these limitations, this research investigates ML models that maintain the performance observed in the literature, but the dataset is adapted to a variable environment, using synthetic data for which privacy rules do not need to be applied, and which acts as an additional layer in production web applications.

3. Materials and Methods

To address the RQs, the authors noted that SQLi attacks introduce unusual patterns, which, alongside the standard query, also add a series of expressions that allow unauthorized content retrieval. For the construction of the dataset, the authors proposed using synthetic data in combination with a public dataset [54]. Both datasets have the same initial structure, containing, as the first column, the user input and, as the second column, the status correlated with a malicious or legitimate value for the input. They have the great advantage of lacking sensitive data. Practically, they do not disclose elements of a confidential nature. The synthetic data were generated using the GPT-4 model, version GPT-4o [55]. The generated data included a column of queries and a column of classifications, which helps determine whether the input is malicious or legitimate. The use of synthetic data in this context comes with a series of remarkable advantages, including the ability to generate a balanced dataset. Practically, using the Large Language Model (LLM) tool with the GPT-4 model, 72,304 data points were generated for the malicious category and 17,695 for the legitimate category. In this way, the generated data did not depend on real situations and were fully controlled regarding the type of simulated attack. In other words, the dataset was generated exclusively for SQLi attack types. By applying this synthetic data generation tactic, legal issues related to the exposure of sensitive data collected from real applications, such as usernames, passwords, personal data, or database schemas, were avoided, and there was also no need for confidentiality agreements for using the data in training and evaluating the models.

The dataset used in this study consisted of 90,000 SQL queries. They were obtained by combining synthetic and publicly available data as follows:

- Synthetic data generated 72,304 malicious queries. The results were generated with the GPT-4o model [55].
- A public dataset with 17,695 legitimate queries from the SQL-Injection-Extend dataset (Kaggle) [54].

Synthetic malicious queries were generated using prompt-based generation with GPT-4o to mimic SQLi attack vectors:

- Boolean expressions like 'val' = 'val' OR '1' = '1';
- The obfuscation of comments in expressions like `/**/ OR /**/1 = 1--;`
- Blind SQLi (AND ASCII(SUBSTRING(...)) > 50--);
- Payloads encoded in the format of the following example: `%27%20OR%20%271%27%3D%271;`
- UNION-based extractions (UNION SELECT password FROM users).

Fixed random seeds were used in the query generation templates to maintain reproducibility. This way, the results remain stable throughout the executions.

The dataset contains atypical structures that highlight malicious patterns. Among these, the authors noted Unicode and URL encoding (e.g., username = `%27%20OR%20%271%27%3D%271`), comments inserted between words (SELECT * FROM test WHERE username = 'admin'/**/OR/**/'1' = '1'), spaces, and special characters (`/**/OR/**/1 = 1--+`), blind attacks (AND ASCII(SUBSTRING((SELECT version()), 1, 1)) > 50--), and the UNION and encode variant (`'union%0aSELECT%201,%20password%20FROM%20users--`). These input examples generate malicious SQLi attacks. On the other hand, the model trained on the synthetic dataset also includes examples that are similar to SQLi attacks but are legitimate.

For the development of the automatic layer for detecting malicious SQL queries, it was necessary to build a software pipeline for preprocessing and feature extraction from SQL queries. Within the pipeline, preprocessing involved a syntactic normalization of the

SQL queries. In the first stage of the pipeline, a series of processing steps were carried out, as follows:

- The conversion to lowercase was performed to standardize the SQL expressions and reduce query variability;
- The values between the apostrophes were replaced with the token VAL in order to generalize the static values and eliminate the unique signatures;
- The numbers were replaced with the token NUM to normalize the numerical expressions;
- The names of tables and columns were generalized, and the identifiers in the SQL expressions were preserved;
- Multiple spaces have been removed to optimize the syntactic structure of the query.

This SQL query normalization stage in the software pipeline prevents overfitting, helping the model learn from semantic patterns rather than custom expressions. This stage is essential for generalizing to new attacks written in different ways.

In the second stage of the software pipeline, a series of semantic features was proposed by the authors as being relevant for identifying and encoding the distinctive elements between legitimate and malicious queries. Table 1 presents the features extracted from the queries processed in the first stage of the pipeline. These features are binary or numerical, and their main objective is to capture the behavior of SQLi characteristics within ML algorithms. These algorithms learn structural and semantic differences between the two classes, legitimate and malicious. They allow a simplified representation of queries, which can be correctly understood and interpreted by ML models. Table 1 presents the 7 extracted features along with the descriptions corresponding to the identified cyberattack typology on SQL queries. Through these stages associated with the software pipeline, the preprocessing of SQL queries was carried out, transforming raw inputs into semantic and syntactic representations optimized for the training stage of SQLi detection algorithms. The process maximizes the performance of ML models, minimizes overfitting, facilitates generalization to unknown or evasive attacks, and prevents the unauthorized use of data during the training phase.

Table 1. Extracted features for SQLi detection.

Feature	Description
<i>has_or</i>	Checks for the presence of the OR operator, commonly used in Boolean-based SQLi attacks.
<i>has_true_pattern</i>	Detects expressions like <code>1 = 1</code> or <code>'val' = 'val'</code> that indicate always true conditions.
<i>has_comment</i>	Checks for the use of SQL comments (<code>--</code> , <code>/* */</code>) used to ignore the remainder of a legitimate query.
<i>has_union</i>	Detects the use of UNION, a common SQLi technique to extract data from other tables.
<i>num_conditions</i>	Counts filtering conditions (<code>WHERE COL = . . .</code>), relevant for analyzing logical structure.
<i>query_length</i>	Length of the normalized query, often correlated with more complex attacks.
<i>has_always_true</i>	Explicitly checks for expressions like <code>'val' = 'val'</code> , a sign of condition manipulation.
<i>has_exec</i>	Detects dangerous commands (<code>EXEC</code> , <code>xp_cmdshell</code>), indicative of advanced exploitation.

The eight features proposed by the authors are motivated by the tactics used in SQLi attacks. The first feature indicates whether the query contains the logical operator OR. This feature is motivated by the fact that SQLi attacks frequently contain the OR operator (*has_or*) in payloads to manipulate WHERE conditions, forcing the return of unwanted results.

The *has_true_pattern* feature detects common patterns that always return true. These expressions are used in SQLi attacks at the web application level. The main objective of this behavior is to unintentionally execute code snippets. For example, including the sequence `'val' = 'val'` in a WHERE clause with multiple alternative conditions will allow access without authentication, as one of the conditions is met.

The third feature, *has_comment*, is justified by the fact that SQLi attacks include comments that freeze the original part of the query and introduce malicious code. The next features presented in Table 1, *has_union* and *num_conditions*, represent the number of conditions in the WHERE clause and were introduced because normal queries usually have few conditions, while malicious queries can include a larger volume of conditions to manipulate the application's logic. A higher value reflects, in the authors' opinion, an attempt at logical bypass.

The sixth feature, *query_length*, reflects the total length of the SQL query, as SQLi attacks tend to include complex payloads. An unusually long length is associated by the authors with an alarm signal that could identify a malicious query. The seventh feature, *has_always_true*, detects the implicit presence of the expression 'val' = 'val' at the query level. This is a particular case, frequently used in SQLi, where equality ensures the truth value of the expression to bypass authorization logic.

The eighth feature, *has_exec*, checks if the query contains calls to dangerous functions such as execution or calls to stored procedures. These represent advanced levels of SQLi attacks, being capable of executing code on the underlying server. Additionally, the presence of these keywords represents a warning signal, and for these reasons, the authors included this feature in the model to be trained.

Figure 1 presents the logical schema corresponding to the two stages of the software pipeline in which the original query is transformed into a series of features that will be used in the ML model training stage. This logical diagram, in Figure 1, was implemented in C# using the Visual Studio Version 17.12.4 development environment.

After implementing the pipeline and running it, a selection of ML techniques was made that fit within the context of SQLi attack detection, as part of an advanced and tailored defensive approach in cybersecurity. These techniques aim to complement traditional methods through an additional layer that detects, behaviorally analyzes, classifies, and automatically decides whether the query is legitimate or malicious. As a consequence, the problem was modeled using ML methods within the Microsoft Azure platform. It provides an extensive class of sampling methods for splitting the dataset into training and validation data, as well as an extensive class of ML training methods.

The project for the automatic training of the ML model was carried out using the Azure Machine Learning Studio platform, with 80% of the dataset used for training and the remaining 20% for validation and testing. In the training stage, several ML algorithms for SQLi detection were evaluated based on the previously preprocessed synthetic data. These models used classic algorithms such as RF, Extra Trees (ET), XGBoost Classifier, LR, LightGBM, and Voting Ensemble. These models were trained to identify the degree to which they can distinguish between the two classes, malicious and legitimate.

The Azure Machine Learning Studio component uses specialized algorithms to transform the initial data to bring it to a common scale as an additional data preprocessing measure. In this research, three scaling algorithms represented by MaxAbsScaler, StandardScalerWrapper, and SparseNormalizer were used. The three scaling algorithms combined with the classification algorithms generate different performances for the trained models.

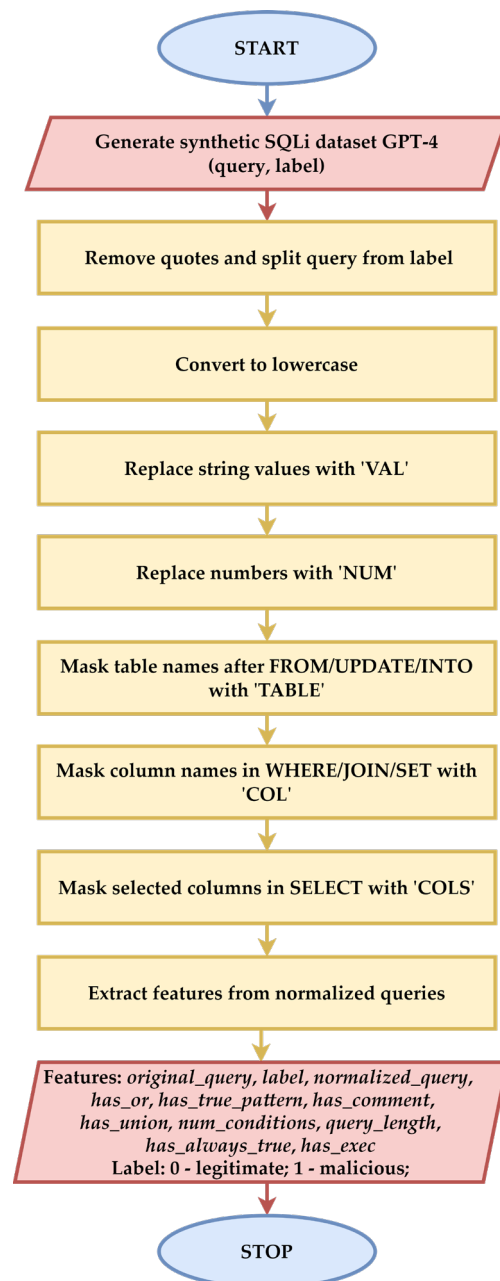


Figure 1. Two-stage preprocessing pipeline for SQLi dataset preparation.

4. Results

In the experiment conducted with Azure Machine Learning Studio, we used a database that contained 90,000 records. The dataset was automatically split by the platform into two subsets. The first subset retained 80% of the records for training, and the remaining 20% for testing. Thus, 72,000 records were used to train the model. The remaining 18,000 were reserved for the final testing of the model.

In the training stage, out of the total records, only 90% are actually used for training, while the remaining 10% are used for internal model validation, based on which performance metrics are generated. These are used to perform a comparative analysis between the ML models evaluated based on performance metrics. Therefore, out of a total of 72,000 records, only 64,800 were actually used for training, while the remaining 7200 were used for internal model validation. Figure 2 presents an example where a query is transformed into features through the two stages of the proposed software pipeline.

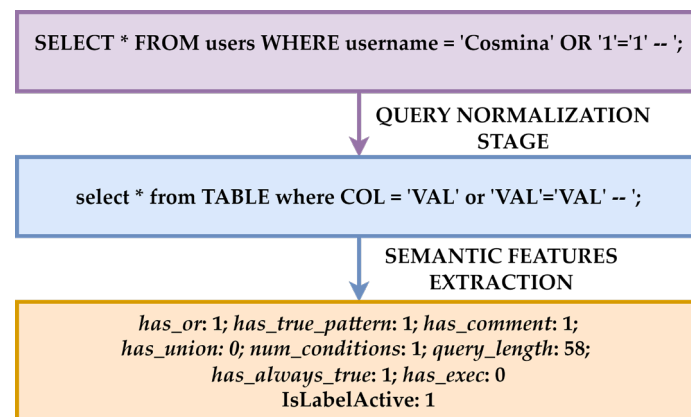


Figure 2. Example of SQL query processing through the proposed pipeline.

Subsequently, the dataset comprising the 72,000 records was trained using Azure Machine Learning Studio. Within this tool, several ML algorithms that used different sampling methods were analyzed, with the combinations of algorithms that provided an Area Under Curve (AUC) of over 90% being retained in Figure 3. Thus, the Voting Ensemble had a weighted AUC of 98.25%. Therefore, this method is considered to have the best results, which is why it was used in future analyses. Based on the model selection process, Voting Ensemble had the best performance, this algorithm being composed of a weighted combination of boosting-type algorithms, with the internal configuration consisting of the LightGBM and XGBoost Classifier algorithms.

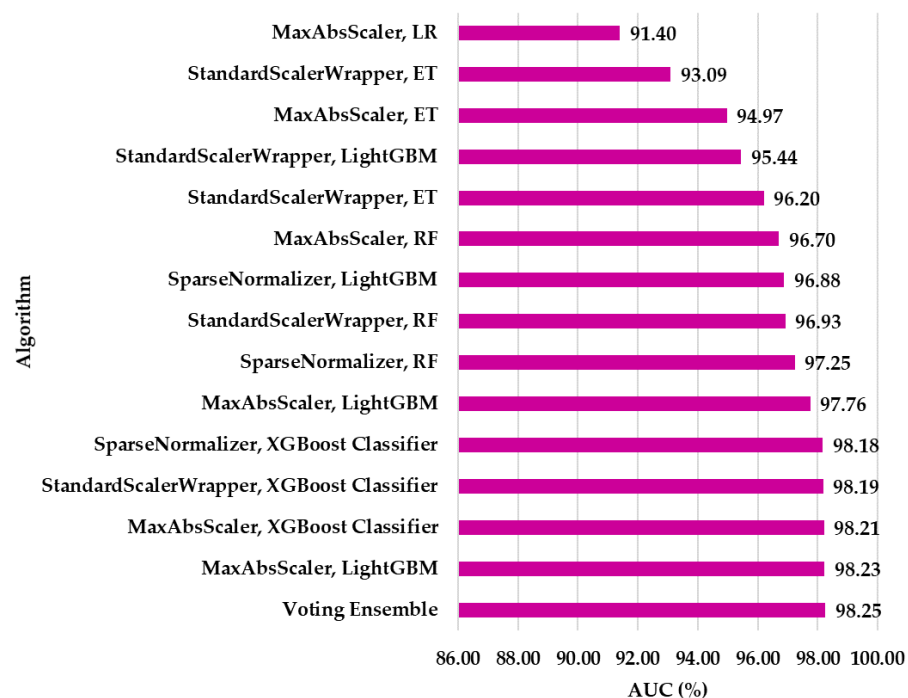


Figure 3. AUC performance metric Azure Machine Learning Studio evaluated models in SQLi detection.

This combination reflects the XGBoost classifier as being dominant, as it appears four times in the composition, complemented by the LightGBM model. This combination of the AutoML pipeline consists of boosting-based decision trees, a result that is confirmed in the context of binary and numerical feature data present at the level of features extracted using the proposed data preprocessing software pipeline. Table 2 presents the

hyperparameters used by the Voting Ensemble model along with the values associated with each hyperparameter.

Table 2. Hyperparameter configuration of the Voting Ensemble model provided by Azure Machine Learning Studio.

Hyperparameter	Description
booster	gbtree
tree_method	hist
grow_policy	lossguide
max_depth	4
max_bin	1023
colsample_bytree	0.7
subsample	0.5
eta	0.3
reg_alpha, reg_lambda	(1.35, 1.87)
n_estimators	100

The Voting Ensemble model is based on the XGBoost Classifier, which has been hyperparameterized to achieve a superior AUC compared to subsequent models, which can be seen in Figure 3 as being represented by the XGBoost Classifier models with different sampling algorithms. Hyperparameter tuning uses Gradient Boosting Decision Trees (GBDT), as shown in Table 2. The algorithm used for histogram construction is indicated through the tree_method parameter, which is set to the value hist. The tree expands in the direction of maximum loss, with the grow_policy parameter set to lossguide. The exploration depth is set to 4 to prevent the risk of overfitting, and the number of bins for the histogram is set to 1023. Subsampling is set to 70%, and data for each tree is sampled at a rate of 50%. The learning rate is set to 30%, and the L1 and L2 regularizations have values of 1.35 and 1.87. The number of trees combined for fast learning is set by the n_estimators parameter, which is set to 100. The values were provided during the hyperparameter tuning process, carried out using the Azure Machine Learning Studio tool.

The performance metrics obtained during internal validation stages are analyzed in the following. This stage corresponds to the training process. Table 3 presents the macro, micro, and weighted values of AUC, average precision, F1-score, precision, and recall (sensitivity).

Table 3. Performance metrics of the trained Voting Ensemble model.

Metric	Macro (%)	Micro (%)	Weighted (%)
AUC	98.25	99.25	98.25
Average precision	97.37	99.26	98.68
F1-score	94.75	96.86	96.77
Precision	97.62	96.86	96.92
Recall (sensitivity)	92.42	96.86	96.86

The AUC parameter measures the model's ability to separate classes, with a value of 99.25% (weighted), confirming that the model has the ability to differentiate between dangerous and legitimate queries. Additionally, the weighted F1-score, representing the harmonic mean between precision and recall, achieved a value of 96.77%, reflecting a balance between the number of correct alerts (precision) and the ability to detect all real attacks (recall). The close values for the micro F1-score and macro F1-score indicate that the model performs well both globally and for each individual class. Weighted precision scored 96.92%, showing that the model correctly flags a query as malicious in the vast majority

of cases. At the same time, the weighted recall achieved a value of 96.86%, indicating that the model successfully identifies almost all dangerous queries with a low number of missed cases (false positives—FPs). The average precision scores (macro, micro, and weighted) were over 97%, confirming that the model has the ability to maintain a high level of performance regardless of the data distribution or the chosen decision thresholds.

The performance of the Voting Ensemble model in the training process can also be measured with additional metrics, such as accuracy, balanced accuracy, weighted accuracy, Matthews correlation coefficient, normalized macro recall, and Log Loss (Table 4). The trained model demonstrates an overall accuracy of 96.86%. This value indicates that the model correctly classifies the majority of SQL queries, whether they are malicious or legitimate.

Table 4. Additional performance metrics of the trained Voting Ensemble model.

Metric	Value (%)
Accuracy	96.86
Balanced accuracy	92.42
Weighted accuracy	98.91
Matthews correlation coefficient	89.89
Normalized macro recall	84.84
Log Loss	10.91

Balanced accuracy has a value of 92.42%, which means that the model maintains performance even under conditions of class imbalance, a situation frequently encountered in real cybersecurity data, where attacks are much rarer than legitimate traffic. Another performance indicator is the Matthews correlation coefficient, which achieved a score of 89.89%. This indicator is a measure of the balance between the correlation of predicted values and actual values. A value close to 100%, as is the case here, suggests a strong correlation. The normalized macro recall of 84.84% shows a slight imbalance in recall across classes, but still within acceptable bounds. The Log Loss score penalizes uncertain predictions, which is why the value is very low, at only 10.91%. This value indicates that the model is reliable in making its own decisions, having a well-balanced probability estimate.

In Figure 4, the confusion matrix of the classification model is presented, which is interpreted as follows:

- 1205 legitimate queries of the dataset are correctly classified as not being attacks (true negatives—TNs).
- 211 legitimate queries are incorrectly classified as attacks (false alarms, labeled as false positive—FPs).
- 15 attacks that were not detected, therefore were incorrectly classified as safe (false negatives—FNs).
- 5769 attacks were correctly detected by the model (true positives—TPs).

TP: 5769	FN: 15
FP: 211	TN: 1205

Figure 4. Confusion matrix of the Voting Ensemble model (the intensity of the background color is directly correlated with the value of the metric).

The values presented in the confusion matrix show a very high detection rate, given that only 15 out of 7200 attacks were missed. This demonstrates the capability of the Voting Ensemble model to be integrated into applications that require layers of cybersecurity to prevent SQLi attacks. The low number of false alarms, 211 out of 7200, indicates a compromise between precision and recall, with a very good value. The ratio of true positives to false positives suggests that the precision is one that allows the integration of the security layer into cyberattack applications. The confusion matrix shows the generalization capability of the model using the Voting Ensemble algorithm. It also confirms through global metrics that the model can act as an automatic threat detection layer.

Figure 5 presents the calibration curve where it can be observed that for predictions with values below 0.5, the model underestimates the probabilities. For high values, above 0.7, the predictions are over-calibrated. Analyzing Figure 5, it is observed that the majority of the values are between 0.8 and 1, demonstrating that the model is very well calibrated. Analyzing this result, along with the calibration curve, it is deduced that the model has a high level of confidence in positive predictions.

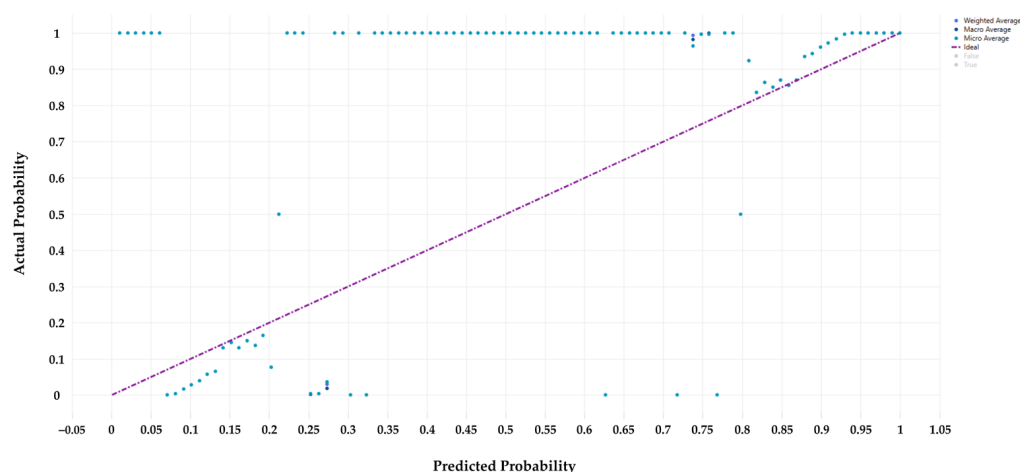


Figure 5. Calibration curve of the Voting Ensemble model.

Figure 6 presents the receiver-operating characteristic (ROC) curve. The macro, micro, and weighted curves are very close to the top left corner, indicating exceptional performance, very close in value to the ideal. The area under the ROC curve is very large, as the reported values are approximately 98%, which confirms the model's ability to discriminate between classes.

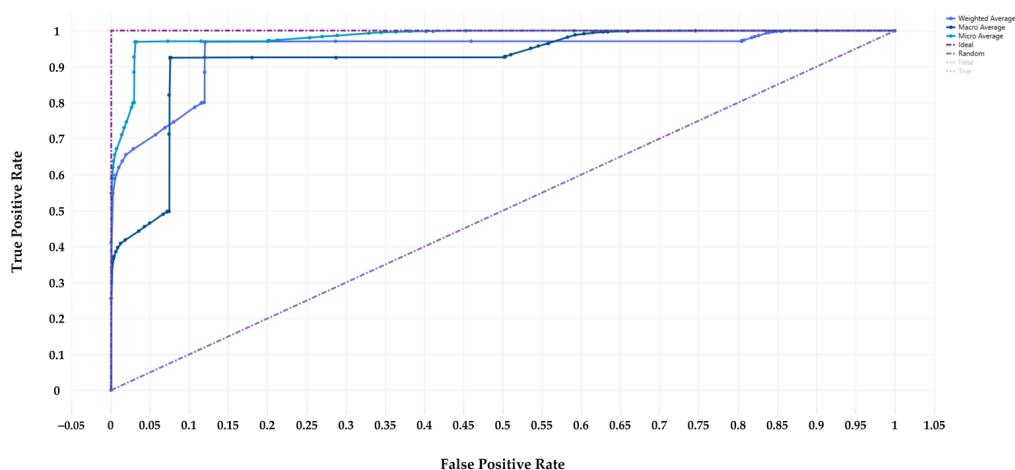


Figure 6. ROC curve of the Voting Ensemble model.

Figure 7 illustrates the cumulative confidence curve, which shows that the trained model outperforms a random model. The high percentages indicate that most curves (weighted average, macro average, and micro average) reach nearly 100%. These values indicate the model's ability to identify positive cases associated with SQLi. Thus, the model can identify positive cases in the vast majority of instances.

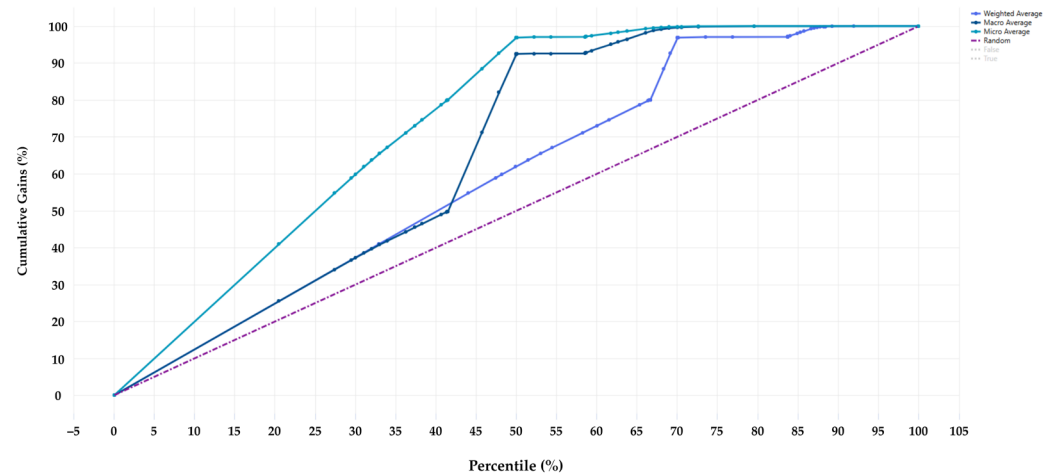


Figure 7. Cumulative gains curve of the Voting Ensemble model.

Figure 8 presents the lift curve. This shows how much the model can improve performance compared to a random selection. The low percentages below 50% indicate that all curves have high values, for which the macro average reaches 3.2. This value indicates an improvement over random selection. As the percentage increases, the curves decline slowly, but remains higher than the random line. As a result, the model has a performance that can identify positive cases, even when the precision slightly decreases at higher levels.

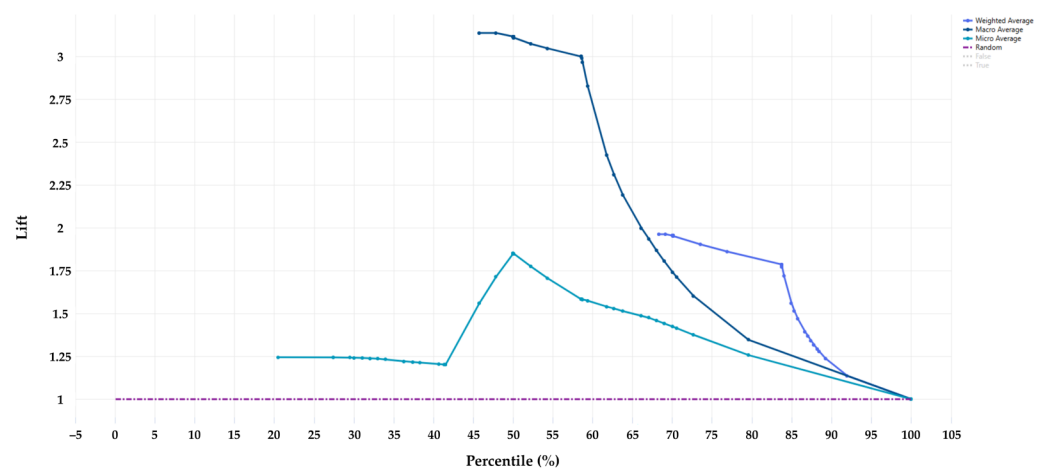


Figure 8. Lift curve of the Voting Ensemble model.

Figure 9 presents the precision–recall curve. This indicates the relationship between the precision and recall of the model. Thus, most of the curves are very close to the ideal line, achieving a precision between 95% and 97%. Therefore, the model offers a combination that suggests a very good balance between the number of correct alerts and the model's ability to detect all real attacks.

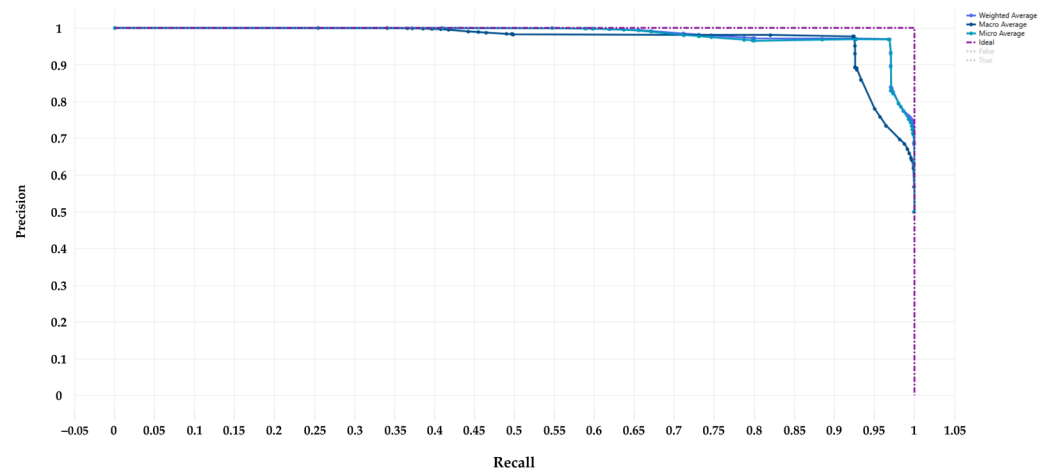


Figure 9. Precision vs. recall of the Voting Ensemble model.

In the validation stage, 18,000 records were used. For the Voting Ensemble model, the following performance metrics were obtained: an accuracy of 96.65%, a precision of 99.47%, an F1-score of 97.95%, a Log Loss of 18.86%, a Matthews correlation coefficient of 89.18%, and a binary recall of 99.63%. These results suggest the model's ability to generalize in the context of cyberattacks on relational databases.

5. Discussion

In this research, ML models for detecting SQLi attacks were studied. The study was based on identifying potential attacks using the analysis of a set of features proposed by the authors.

The authors implemented an additional ablation analysis to evaluate the contribution of each semantic feature to the model's performance. In each iteration, one feature is removed for comparative accuracy evaluation. The results are presented in Table 5.

Table 5. Analysis of feature contribution through the ablation study.

Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7	Accuracy (%)
<i>has_or</i>	<i>has_true_pattern</i>	<i>has_comment</i>	<i>num_conditions</i>	<i>query_length</i>	<i>has_always_true</i>	<i>has_exec</i>	96.86
-	<i>has_true_pattern</i>	<i>has_comment</i>	<i>num_conditions</i>	<i>query_length</i>	<i>has_always_true</i>	<i>has_exec</i>	96.66
<i>has_or</i>	-	<i>has_comment</i>	<i>num_conditions</i>	<i>query_length</i>	<i>has_always_true</i>	<i>has_exec</i>	96.7
<i>has_or</i>	<i>has_true_pattern</i>	-	<i>num_conditions</i>	<i>query_length</i>	<i>has_always_true</i>	<i>has_exec</i>	95.23
<i>has_or</i>	<i>has_true_pattern</i>	<i>has_comment</i>	-	<i>query_length</i>	<i>has_always_true</i>	<i>has_exec</i>	96.47
<i>has_or</i>	<i>has_true_pattern</i>	<i>has_comment</i>	<i>num_conditions</i>	-	<i>has_always_true</i>	<i>has_exec</i>	80.54
<i>has_or</i>	<i>has_true_pattern</i>	<i>has_comment</i>	<i>num_conditions</i>	<i>query_length</i>	-	<i>has_exec</i>	96.69
<i>has_or</i>	<i>has_true_pattern</i>	<i>has_comment</i>	<i>num_conditions</i>	<i>query_length</i>	<i>has_always_true</i>	-	96.7

The largest performance decrease is associated with the *query_length* feature. In the scenario where *query_length* is removed, the accuracy decreased from 96.86% to 80.54%. Thus, it is found that the normalized query length is an indicator that contributes to the detection of SQLi attacks. This is explained by the fact that malicious payloads are longer than legitimate queries.

Removing the *has_or* feature led to a decrease from 96.86% to 96.70%. It can also be seen in Table 5 that removing *has_true_pattern* resulted in 96.66%. The smallest effects were observed for *has_exec* and *has_always_true*.

This analysis demonstrates that the proposed features contribute to the model's performance in making correct classifications.

The authors emphasize that these characteristics have been extensively studied in the literature for SQLi detection. The novelty lies in the proposed software infrastructure for identifying attacks. The two-stage software pipeline proposed in this paper is considered by the authors to be the novel element that is the reference for this research.

The ML methods were studied using the Azure Machine Learning tool, and out of all of the analyzed models, Voting Ensemble provided the best results, achieving an accuracy of 96.86% and a weighted AUC of 98.25%. These values indicate that the model using Voting Ensemble can distinguish between legitimate and malicious queries.

These results answer the RQs as follows:

- RQ1: The dataset was constructed using synthetic data generated with the help of the GPT-4o model. This dataset complies with GDPR regulations by providing data that can be used for model training. Additionally, the dataset also contains real data sourced from a public dataset. The dataset consisted of 90,000 SQL queries, which were processed in a software pipeline proposed by the authors. The pipeline contained two stages: the first was data normalization through replacement with standard expressions, and the second was feature extraction from these expressions. Feature extraction is justified by the need to train the model based on repetitive elements, rather than direct training using the SQL query.
- RQ2: The set of features used was proposed by the authors to extract common behaviors in SQLi attacks. The features they proposed are as follows: the OR operator, always true conditions, the presence of SQL comments, unions, the number of conditions, the length of the query, TRUE ALWAYS expressions, or EXEC calls. These features have served as reference elements in building the dataset used for training the ML models.
- RQ3: The feasibility of integrating the ML layer is supported as an intermediate layer between input entry and the actual execution of the query. The authors emphasize that this layer does not replace traditional ORM measures or Prepared Statements.
- RQ4: The obtained metrics indicate values that allow the practical integration of this intermediate layer. For the Voting Ensemble model, the metrics were a weighted F1-score of 96.77%, a balanced accuracy of 92.42%, a Matthews correlation of 89.89%, an overall accuracy of 96.86%, and a weighted AUC of 98.25%.

These correlations, identified between data features and malicious behaviors, have allowed for the segmentation between SQLi queries. The resulting model outperforms a random system with a 3.2% lift according to the dedicated curve. The ROC curve and the calibration curve show that the model is well-fitted, having high confidence in its predictions for the positive classes.

These results confirm the hypothesis that SQLi is detected through the behavioral analysis of semantic patterns. This remark, supported by the demonstration through performance metrics, represents a contribution by the authors to the specialized literature, where traditional approaches focus more on static filtering and fixed rules, without considering modern techniques that can reduce the possibilities of SQLi injection to zero.

An important aspect of the low FNs, represented by only 15 out of 7200 attacks, is that the risk of allowing access to dangerous queries is minimal. Moreover, the 15 cases are subsequently filtered in the layer represented by ORM or Prepared Statements. The proposed model for integration into an additional layer of security in web applications contributes to the prevention of real-time attacks, without replacing classical approaches, but rather complementing them with a new security contribution.

The study presents, as limitations, the use of only eight features, with the possibility of adding other characteristics that could be correlated with subtle signals for detecting advanced attacks.

The model was evaluated offline using the synthetic and public datasets. This has not been directly tested in a production environment. The results obtained demonstrate the potential for integration into practice with the help of performance metrics. The authors recall that the model was developed within the Microsoft Azure platform, which offers an advantage regarding its implementation in a web architecture. This is possible because applications that already use the Azure ecosystem are compatible with all other cloud-native solutions on the same platform. The model can be exposed in a web service through a REST endpoint. As a future direction, the authors propose to implement a functional prototype in an Azure environment using a Web application. This will allow us to measure the request processing rate and the model's behavior under real traffic conditions. The application using the model requires years of maturity to measure performance in production, as SQLi attacks are not daily occurrences, which makes testing it in production for performance reasons unfeasible.

A primary limitation of this article is related to the small number of semantic features obtained from the synthetic dataset generated using the GPT-4o model, which was subsequently supplemented with data from a public dataset. Beyond the advantages associated with GDPR regulations and data distribution methods, the authors acknowledge the risk that the model may not generalize perfectly in real-world environments where attackers use evasive techniques, complex obfuscation, new combinations of payloads, or techniques that were not yet known to experts in the field. Validation on real, anonymized traffic collected from production applications in real scenarios with real users is a future step for this model.

Another limitation of the study is that the paper does not include tools for interpreting the model's decisions for real users. Examples of these are developers or system administrators. Although the semantic features used are interpretable at the model level, the lack of an interface or mechanism to clearly explain why a specific query was classified as malicious is a limitation of this study.

The dataset used in this research is not exclusively synthetic. This represents a combination of the dataset generated with the GPT-4o model and a public dataset [54]. However, the authors acknowledge that the public dataset used is not a standardized benchmark from the literature, like the one described by Paul et al. [27]. Currently, the model is not evaluated on such a benchmark. Therefore, this is a limitation in terms of direct behavior with the state-of-the-art. As a future direction, the authors will validate the proposed model on standard public benchmarks, such as those from the research by Paul et al. [27]. The authors also want to study the collection and anonymized use of data from production environments so they can test the model against evasive, obfuscated, and adaptive attacks. These steps will allow the model to function in scenarios aligned with current security practices.

Based on the discussions, the authors propose in the future to expand the feature set with new behavioral dimensions, such as the frequency of similar queries reported over a specific time interval, the use of real data collected anonymously under variable and multi-user traffic conditions, as well as the development of a hybrid system that combines ML prediction with Intrusion Detection System signatures and studies the impact of this system on the web application's performance from a response time perspective.

6. Conclusions

The evolution of digital technologies has increased the rate of SQLi vulnerabilities, necessitating further studies aimed at the possibility of integrating an additional layer of security. The authors of this study recommend integrating traditional solutions alongside this additional layer that includes ML technology and whose objective is the automatic

detection of malicious SQL queries. To address this challenge, the study analyzed a dataset composed of synthetic data and real data from a public dataset.

The first contribution of this study specifically addresses the advantages of using synthetic datasets in the study of ML techniques. Subsequently, repetitive features for SQLi attacks were proposed and extracted from the constructed dataset. These features were trained within various models that were paired with different sampling methods. Thus, 15 combinations of training algorithm sampling resulted. The 15 combinations were trained using the Azure Machine Learning tool. Out of all of the combinations, the Voting Ensemble model provided the best performance metrics. This algorithm belongs to the class of boosting algorithms, achieving an accuracy of 96.86%, with a weighted AUC of 98.25%. These values confirm the model's ability to differentiate between the two classes.

In this paper, a comprehensive methodology was also proposed for the generation, processing, and handling of data for the purpose of training an SQLi detection model and extracting the semantic features used in query discrimination. These elements were validated and analyzed comparatively in terms of the performance of the 15 training-sampling algorithm combinations.

Also within this work, the Voting Ensemble algorithm was identified as having the best performance. The paper demonstrates that integrating this model into an intermediate security layer enhances the security of a web application, as it adds an additional layer of protection alongside traditional methods. The results of this research have practical implications for web applications and are intended for developers concerned with the security of real-time applications.

A theoretical contribution of the research is the validation of using synthetic data in training ML models, and the research demonstrates that this is possible without compromising the model's performance. In this paper, a limitation can be identified as the lack of testing in production environments. For this reason, future research will explore the expansion of the dataset with anonymized real data. Thus, the authors aim to create an algorithm that will anonymize this data at a higher level of difficulty compared to that proposed in this study. The creation of hybrid models that combine ML algorithms with static system signatures and the analysis of the impact on the performance of the web application, which is integrated as an additional layer, represent other future research directions for the authors.

The research supports the idea that SQLi attack detection can be achieved by combining traditional methods with ML techniques. The authors recommend that organizations consider developing web applications that also integrate this additional layer of security protection against modern cyber threats.

Author Contributions: Conceptualization, C.-M.R.; methodology, C.-M.R., A.S., and C.P.; software, C.-M.R. and A.S.; validation, C.-M.R., A.S., and C.P.; formal analysis, C.-M.R., A.S., and C.P.; investigation, C.-M.R., A.S., and C.P.; resources, C.-M.R., A.S., and C.P.; data curation, C.-M.R. and A.S.; writing—original draft preparation, C.-M.R., A.S., and C.P.; writing—review and editing, C.-M.R., A.S., and C.P.; visualization C.-M.R., A.S., and C.P.; supervision, C.-M.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Petroleum-Gas University of Ploiesti, Romania.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AdaBoost	Adaptive Boosting
AI	Artificial Intelligence
AST	Abstract Syntax Tree
AUC	Area Under Curve
CNN	Convolutional Neural Network
DL	Deep Learning
ET	Extra Trees
FN	False Negative
FNN	Fuzzy Neural Network
FP	False Positive
FST	Feature Selection Technique
GBDT	Gradient Boosting Decision Trees
GDPR	General Data Protection Regulation
KMP	Knuth–Morris–Pratt
LightGBM	Light Gradient Boosting Machine
LLM	Large Language Model
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PNN	Probabilistic Neural Network
RF	Random Forest
RL	Reinforcement Learning
RNN	Recurrent Neural Network
ROC	Receiver-operating Characteristic
RQ	Research Questions
RUS	Random Undersampling
SDN	Software-Defined Networking
SQL	Structured Query Language
SQLi	SQL Injection
TN	True Negative
TP	True Positive
WAFs	Web Application Firewalls
XGBoost	Extreme Gradient Boosting

References

1. Alhowiti, A.H.; Mohamed, A.M.A. A New Database Integrity Protection Approach Against SQL Injection Attacks (SQLIAs). In Proceedings of the 4th International Conference on Computing and Information Technology, Tabuk, Saudi Arabia, 13–14 April 2025; pp. 507–512. [\[CrossRef\]](#)
2. Jang, Y.-S. Detection of SQL Injection Vulnerability in Embedded SQL. *IEICE Trans. Inf. Syst.* **2020**, *E103.D*, 1173–1176. [\[CrossRef\]](#)
3. Rosca, C.-M.; Stancu, A. A Comprehensive Review of Machine Learning Models for Optimizing Wind Power Processes. *Appl. Sci.* **2025**, *15*, 3758. [\[CrossRef\]](#)
4. Yuan, Y.; Lu, Y.; Zhu, K.; Huang, H.; Yu, L.; Zhao, J. A Static Detection Method for SQL Injection Vulnerability Based on Program Transformation. *Appl. Sci.* **2023**, *13*, 11763. [\[CrossRef\]](#)
5. Erdödi, L.; Sommervoll, Å.Å.; Zennaro, F.M. Simulating SQL injection vulnerability exploitation using Q-learning reinforcement learning agents. *J. Inf. Secur. Appl.* **2021**, *61*, 102903. [\[CrossRef\]](#)
6. Abikoye, O.C.; Abubakar, A.; Dokoro, A.H.; Akande, O.N.; Kayode, A.A. A novel technique to prevent SQL injection and cross-site scripting attacks using Knuth-Morris-Pratt string match algorithm. *EURASIP J. Inf. Secur.* **2020**, *2020*, 14. [\[CrossRef\]](#)

7. Bedeković, N.; Havaš, L.; Horvat, T.; Crčić, D. The Importance of Developing Preventive Techniques for SQL Injection Attacks. *Tehnički Glasnik* **2022**, *16*, 523–529. [\[CrossRef\]](#)
8. Yang, G.-Y.; Wang, F.; Gu, Y.-Z.; Teng, Y.-W.; Yeh, K.-H.; Ho, P.-H.; Wen, W.-L. TPSQLi: Test Prioritization for SQL Injection Vulnerability Detection in Web Applications. *Appl. Sci.* **2024**, *14*, 8365. [\[CrossRef\]](#)
9. Alghawazi, M.; Alghazzawi, D.; Alarifi, S. Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review. *J. Cybersecur. Priv.* **2022**, *2*, 764–777. [\[CrossRef\]](#)
10. Farooq, U. Ensemble Machine Learning Approaches for Detection of SQL Injection Attack. *Tehnički Glasnik* **2021**, *15*, 112–120. [\[CrossRef\]](#)
11. Shareef, O.S.F.; Hasan, R.F.; Farhan, A.H. Analyzing SQL payloads using logistic regression in a big data environment. *J. Intell. Syst.* **2023**, *32*, 20230063. [\[CrossRef\]](#)
12. Rosca, C.-M.; Stancu, A.; Iovanovici, E.M. The New Paradigm of Deepfake Detection at the Text Level. *Appl. Sci.* **2025**, *15*, 2560. [\[CrossRef\]](#)
13. Arasteh, B.; Bouyer, A.; Sefati, S.S.; Craciunescu, R. Effective SQL Injection Detection: A Fusion of Binary Olympiad Optimizer and Classification Algorithm. *Mathematics* **2024**, *12*, 2917. [\[CrossRef\]](#)
14. Arasteh, B.; Aghaei, B.; Farzad, B.; Arasteh, K.; Kiani, F.; Torkamanian-Afshar, M. Detecting SQL injection attacks by binary gray wolf optimizer and machine learning algorithms. *Neural Comput. Appl.* **2024**, *36*, 6771–6792. [\[CrossRef\]](#)
15. Le, T.-T.-H.; Hwang, Y.; Choi, C.; Wardhani, R.W.; Putranto, D.S.C.; Kim, H. Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and Boosting Models Using Local Explanation Techniques. *Electronics* **2024**, *13*, 4350. [\[CrossRef\]](#)
16. Peralta-Garcia, E.; Quevedo-Monsalbe, J.; Tuesta-Montez, V.; Arcila-Diaz, J. Detecting Structured Query Language Injections in Web Microservices Using Machine Learning. *Informatics* **2024**, *11*, 15. [\[CrossRef\]](#)
17. Banimustafa, A.; Baklizi, M.; Khatatneh, K. Machine Learning for Securing Traffic in Computer Networks. *Int. J. Adv. Comput. Sci. Appl.* **2022**, *13*, 426–435. [\[CrossRef\]](#)
18. Rosca, C.-M.; Stancu, A. Quality assessment of GPT-3.5 and Gemini 1.0 Pro for SQL syntax. *Comput. Stand. Interfaces* **2026**, *95*, 104041. [\[CrossRef\]](#)
19. Thalji, N.; Raza, A.; Islam, M.S.; Samee, N.A.; Jamjoom, M.M. AE-Net: Novel Autoencoder-Based Deep Features for SQL Injection Attack Detection. *IEEE Access* **2023**, *11*, 135507–135516. [\[CrossRef\]](#)
20. Alghawazi, M.; Alghazzawi, D.; Alarifi, S. Deep Learning Architecture for Detecting SQL Injection Attacks Based on RNN Autoencoder Model. *Mathematics* **2023**, *11*, 3286. [\[CrossRef\]](#)
21. Alarfaj, F.K.; Khan, N.A. Enhancing the Performance of SQL Injection Attack Detection through Probabilistic Neural Networks. *Appl. Sci.* **2023**, *13*, 4365. [\[CrossRef\]](#)
22. Muduli, D.; Shookdeb, S.; Zamani, A.T.; Saxena, S.; Kanade, A.S.; Parveen, N.; Shameem, M. SIDNet: A SQL Injection Detection Network for Enhancing Cybersecurity. *IEEE Access* **2024**, *12*, 176511–176526. [\[CrossRef\]](#)
23. Bakır, R. UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques. *Arab. J. Sci. Eng.* **2025**, *1*–14. [\[CrossRef\]](#)
24. Lo, R.-T.; Hwang, W.-J.; Tai, T.-M. SQL Injection Detection Based on Lightweight Multi-Head Self-Attention. *Appl. Sci.* **2025**, *15*, 571. [\[CrossRef\]](#)
25. Rosca, C.-M.; Stancu, A.; Tănase, M.R. A Comparative Study of Azure Custom Vision Versus Google Vision API Integrated into AI Custom Models Using Object Classification for Residential Waste. *Appl. Sci.* **2025**, *15*, 3869. [\[CrossRef\]](#)
26. Tadhani, J.R.; Vekariya, V.; Sorathiya, V.; Alshathri, S.; El-Shafai, W. Securing web applications against XSS and SQLi attacks using a novel deep learning approach. *Sci. Rep.* **2024**, *14*, 1803. [\[CrossRef\]](#)
27. Paul, A.; Sharma, V.; Olukoya, O. SQL injection attack: Detection, prioritization & prevention. *J. Inf. Secur. Appl.* **2024**, *85*, 103871. [\[CrossRef\]](#)
28. Sun, H.; Du, Y.; Li, Q. Deep Learning-Based Detection Technology for SQL Injection Research and Implementation. *Appl. Sci.* **2023**, *13*, 9466. [\[CrossRef\]](#)
29. Crespo-Martínez, I.S.; Campazas-Vega, A.; Guerrero-Higueras, Á.M.; Riego-Delcastillo, V.; Álvarez-Aparicio, C.; Fernández-Llamas, C. SQL injection attack detection in network flow data. *Comput. Secur.* **2023**, *127*, 103093. [\[CrossRef\]](#)
30. Rosca, C.-M.; Stancu, A. Earthquake Prediction and Alert System Using IoT Infrastructure and Cloud-Based Environmental Data Analysis. *Appl. Sci.* **2024**, *14*, 10169. [\[CrossRef\]](#)
31. Alqhtani, M.; Alghazzawi, D.; Alarifi, S. Black-Box Adversarial Attacks Against SQL Injection Detection Model. *Contemp. Math.* **2024**, *5*, 5098–5112. [\[CrossRef\]](#)
32. Demetrio, L.; Valenza, A.; Costa, G.; Lagorio, G. WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning. In Proceedings of the 35th Annual ACM Symposium on Applied Computing, Brno, Czech Republic, 30 March–3 April 2020; pp. 1745–1752. [\[CrossRef\]](#)

33. Valenza, A.; Demetrio, L.; Costa, G.; Lagorio, G. WAF-A-MoLE: An adversarial tool for assessing ML-based WAFs. *SoftwareX* **2020**, *11*, 100367. [\[CrossRef\]](#)
34. Zuech, R.; Hancock, J.; Khoshgoftaar, T.M. A new feature popularity framework for detecting cyberattacks using popular features. *J. Big Data* **2022**, *9*, 119. [\[CrossRef\]](#)
35. Liu, X. Corpus-Based Japanese Reading Teaching Database Cloud Service Model. *Math. Probl. Eng.* **2022**, *2022*, 2011703. [\[CrossRef\]](#)
36. Ismail, S.; Dandan, S.; Dawoud, D.W.; Reza, H. A Comparative Study of Lightweight Machine Learning Techniques for Cyber-Attacks Detection in Blockchain-Enabled Industrial Supply Chain. *IEEE Access* **2024**, *12*, 102481–102491. [\[CrossRef\]](#)
37. Rani, S.V.J.; Ioannou, I.I.; Nagaradjane, P.; Christophorou, C.; Vassiliou, V.; Yarramsetti, H.; Shridhar, S.; Balaji, L.M.; Pitsillides, A. A Novel Deep Hierarchical Machine Learning Approach for Identification of Known and Unknown Multiple Security Attacks in a D2D Communications Network. *IEEE Access* **2023**, *11*, 95161–95194. [\[CrossRef\]](#)
38. Sukhni, B.A.; Manna, S.K.; Dave, J.M.; Zhang, L. Extracting Optimal Number of Features for Machine Learning Models in Multilayer IoT Attacks. *Sensors* **2024**, *24*, 8121. [\[CrossRef\]](#)
39. Rosca, C.-M.; Stancu, A.; Popescu, M. The Impact of Cloud Versus Local Infrastructure on Automatic IoT-Driven Hydroponic Systems. *Appl. Sci.* **2025**, *15*, 4016. [\[CrossRef\]](#)
40. Souza, M.S.; Ribeiro, S.E.S.B.; Lima, V.C.; Cardoso, F.J.; Gomes, R.L. Combining Regular Expressions and Machine Learning for SQL Injection Detection in Urban Computing. *J. Internet Serv. Appl.* **2024**, *15*, 103–111. [\[CrossRef\]](#)
41. Pejó, B.; Kapui, N. SQLi Detection with ML: A Data-Source Perspective. In Proceedings of the 20th International Conference on Security and Cryptography, Rome, Italy, 10–12 July 2023; pp. 642–648. [\[CrossRef\]](#)
42. Maruthavani, E.; Shantharajah, S.P. Real-Time HealthCare Recommendation System for Social Media Platforms. *IEEE Access* **2024**, *12*, 74161–74168. [\[CrossRef\]](#)
43. Ahmed, M.; Altamimi, A.B.; Khan, W.; Alsaffar, M.; Ahmad, A.; Khan, Z.H.; Alreshidi, A. PhishCatcher: Client-Side Defense Against Web Spoofing Attacks Using Machine Learning. *IEEE Access* **2023**, *11*, 61249–61263. [\[CrossRef\]](#)
44. L., R.D.; V., A.H.; Honnavalli, P.B.; S., N. The MongoDB injection dataset: A comprehensive collection of MongoDB-NoSQL injection attempts and vulnerabilities. *Data Brief* **2024**, *54*, 110289. [\[CrossRef\]](#) [\[PubMed\]](#)
45. Bala Gayathri, D.; Sangeetha, D.; Lin, C.; Yu, C.W.; Wang, N. Enhancing security in digitized healthcare system using blockchain technology. *Technol. Health Care* **2024**, *32*, 5105–5127. [\[CrossRef\]](#)
46. Rosca, C.-M.; Bold, R.-A.; Gereia, A.-E. A Comprehensive Patient Triage Algorithm Incorporating ChatGPT API for Symptom-Based Healthcare Decision-Making. In Proceedings of the Emerging Trends and Technologies on Intelligent Systems, ETTIS 2024, Lecture Notes in Networks and Systems, Noida, India, 27–28 March 2024; pp. 167–178. [\[CrossRef\]](#)
47. Ashlam, A.A.; Badii, A.; Stahl, F. A Novel Approach Exploiting Machine Learning to Detect SQLi Attacks. In Proceedings of the 5th International Conference on Advanced Systems and Emergent Technologies, Hammamet, Tunisia, 22–25 March 2022; pp. 513–517. [\[CrossRef\]](#)
48. Zhao, S.; Zhu, J.; Peng, J. Software Vulnerability Mining and Analysis Based on Deep Learning. *Comput. Mater. Contin.* **2024**, *80*, 3263–3287. [\[CrossRef\]](#)
49. Rafique, S.H.; Abdallah, A.; Musa, N.S.; Murugan, T. Machine Learning and Deep Learning Techniques for Internet of Things Network Anomaly Detection—Current Research Trends. *Sensors* **2024**, *24*, 1968. [\[CrossRef\]](#)
50. Ahmad, S.; Zaman, M.; Sami Al-Shamayleh, A.; Ahmad, R.; Muhammad Abdulhamid, S.I.; Ergen, I.; Akhunzada, A. Across the Spectrum In-Depth Review AI-Based Models for Phishing Detection. *IEEE Open J. Commun. Soc.* **2025**, *6*, 2065–2089. [\[CrossRef\]](#)
51. Janabi, A.H.; Kanakis, T.; Johnson, M. Survey: Intrusion Detection System in Software-Defined Networking. *IEEE Access* **2024**, *12*, 164097–164120. [\[CrossRef\]](#)
52. Zuech, R.; Hancock, J.; Khoshgoftaar, T.M. Investigating rarity in web attacks with ensemble learners. *J. Big Data* **2021**, *8*, 71. [\[CrossRef\]](#)
53. Rosca, C.M.; Popescu, M.; Patrascioiu, C.; Stancu, A. Comparative Analysis of pH Level Between Pasteurized and UTH Milk Using Dedicated Developed Application. *Rev. Chim.* **2019**, *70*, 3917–3920. [\[CrossRef\]](#)
54. AlexTrinity. Kaggle. SQL-Injection-Extend. 2022. Available online: <https://www.kaggle.com/datasets/alextrinity/sqlinjectionextend/data> (accessed on 4 June 2025).
55. OpenAI. GPT-4o. Available online: <https://openai.com/index/hello-gpt-4o/> (accessed on 25 June 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.